Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cryptographie appliquée & Blockchain - TP4 - Structure de Feistel et algorithme DES
4ème année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignante : Mme. FADI

Date : 16 octobre 2026

Table des matières

Objectifs **2**

1 Exercice 1 — Le principe de Feistel **3**
1.1 Question 1 . . . 3
1.2 Question 2 . . . 3
1.3 Question 3 . . . 3
1.4 Question 4 . . . 3

2 Exercice 2 — La structure de DES **4**
2.1 Question 1 . . . 4
2.2 Question 2 . . . 4
2.3 Question 3 . . . 4

3 Exercice 3 — Bits de parité **5**
3.1 Question 1 . . . 5
3.2 Question 2 . . . 5
3.3 Question 3 . . . 5

4 Exercice 4 — Exemple simplifié de Feistel **6**
4.1 Question 1 . . . 6
4.2 Question 2 . . . 6
4.3 Question 3 . . . 6

5 Travail personnel — Analyse conceptuelle **7**
5.1 Question 1 . . . 7
5.2 Question 2 . . . 7
5.3 Question 3 . . . 8

Objectifs

— Comprendre la logique et la réversibilité de la structure de chiffrement de Feistel.

— Identifier les principaux composants et le déroulement de l'algorithme DES.

— Expliquer les concepts de **confusion** et de **diffusion** en cryptographie symétrique.

— Analyser l'impact de l'ordre des clés, des permutations et des bits de parité sur le chiffrement/déchiffrement DES.

Consignes

— Le compte rendu doit être déposé une semaine après la séance de TP sur la plateforme Moodle, en respectant la date limite indiquée.

— Le TP doit être réalisé individuellement en classe, mais le compte rendu peut être rendu par groupes de 2 étudiants au maximum.

— Les groupes de TP doivent rester les mêmes pour tous les comptes rendus du semestre.

1 Exercice 1 — Le principe de Feistel

1.1 Question 1

Expliquez la différence entre le chiffrement et le déchiffrement dans un réseau de Feistel. Pourquoi les deux peuvent-ils utiliser la même fonction F ?

Chiffrement : le bloc est coupé en deux moitiés (L_0, R_0), puis on applique n tours avec les sous-clés dans l'ordre $K_1, K_2, \ldots, K_n$.

Déchiffrement : la structure est exactement la même, mais les sous-clés sont appliquées dans l'ordre inverse $K_n, \ldots, K_1$.

Les deux peuvent utiliser la même fonction F car, à chaque tour, F sert uniquement à calculer une valeur qui est combinée par XOR avec l'autre moitié. Le XOR étant sa propre inverse ($A \oplus B \oplus B = A$), il suffit de recalculer la même valeur $F(R, K)$ avec la même entrée et la même clé pour l'annuler. On n'a donc jamais besoin d'inverser F.

1.2 Question 2

Considérons un chiffrement de Feistel à 3 tours défini par :

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

Montrez comment L_0 et R_0 peuvent être retrouvés à partir de L_3 et R_3 en appliquant les clés de tour dans l'ordre inverse.

Pour un tour : $L_i = R_{i-1}$ et $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$. On en déduit $R_{i-1} = L_i$ et $L_{i-1} = R_i \oplus f(L_i, K_i)$.

$$\begin{array}{lll} \text{Tour 3 (avec } K_3) : & R_2 = L_3, & L_2 = R_3 \oplus f(L_3, K_3) \\ \text{Tour 2 (avec } K_2) : & R_1 = L_2, & L_1 = R_2 \oplus f(L_2, K_2) \\ \text{Tour 1 (avec } K_1) : & R_0 = L_1, & L_0 = R_1 \oplus f(L_1, K_1) \end{array}$$

Les clés sont donc utilisées dans l'ordre K_3, K_2, K_1, et l'on retrouve bien (L_0, R_0).

1.3 Question 3

Que se passe-t-il si le processus de déchiffrement utilise les sous-clés dans le même ordre que le chiffrement (c'est-à-dire sans les inverser) ?

Si les sous-clés sont appliquées dans le même ordre qu'au chiffrement (K_1, K_2, K_3), le premier tour de déchiffrement calcule $f(\cdot, K_1)$ alors qu'il devrait annuler $f(\cdot, K_3)$. Les termes XORés au chiffrement ne sont donc pas annulés : chaque tour ajoute une nouvelle valeur au lieu d'en retirer une. Le résultat n'est pas le texte clair (L_0, R_0) mais une valeur incorrecte, et le déchiffrement échoue (sauf cas particulier, par exemple si toutes les sous-clés sont identiques).

1.4 Question 4

Expliquez pourquoi un chiffrement de Feistel est réversible même si la fonction de tour f elle-même n'est pas inversible.

Un tour de Feistel est la transformation $(L, R) \mapsto (R, L \oplus f(R, K))$. Pour l'inverser, il suffit de connaître R (qui est transmis tel quel, donc disponible dans la sortie), de recalculer $f(R, K)$ dans le sens direct, puis de le XORer avec l'autre moitié pour retrouver L. On n'a jamais à calculer f^{-1}.

Chaque tour est donc une permutation bijective quelle que soit f (même non inversible, comme les boîtes S de DES), et la composition de tours reste bijective : le chiffrement est réversible.

2 Exercice 2 — La structure de DES

2.1 Question 1

Quelles sont la taille de bloc et la taille effective de clé utilisées dans DES ?

DES travaille sur des **blocs de 64 bits** (deux moitiés de 32 bits dans le réseau de Feistel). La clé fournie fait 64 bits, mais sa **taille effective est de 56 bits**, car 8 bits sont des bits de parité (voir l'exercice 3). L'espace des clés contient donc 2^{56} clés possibles.

2.2 Question 2

DES applique 16 tours de Feistel en utilisant les sous-clés $K_1, K_2, \ldots, K_{16}$. Comment le processus de déchiffrement utilise-t-il ces clés ?

Le déchiffrement de DES utilise **le même algorithme** que le chiffrement (permutation initiale, 16 tours de Feistel, échange final des moitiés, permutation finale), mais avec les sous-clés appliquées dans **l'ordre inverse** : $K_{16}, K_{15}, \ldots, K_1$. Le premier tour de déchiffrement utilise donc K_{16} et le dernier utilise K_1. Cela permet d'utiliser la même implémentation (logicielle ou matérielle) pour chiffrer et déchiffrer, en changeant seulement l'ordre de présentation des sous-clés.

2.3 Question 3

Décrivez les étapes de la fonction de tour f de DES :

- *Expansion (boîte E)*
- *XOR avec la sous-clé*
- *Substitution (boîtes S)*
- *Permutation (boîte P)*

Expliquez brièvement comment ces étapes contribuent à la confusion et à la diffusion.

La fonction f prend en entrée la moitié droite R_{i-1} (32 bits) et la sous-clé K_i (48 bits) et produit une sortie de 32 bits :

1. **Expansion (boîte E) :** les 32 bits de R_{i-1} sont étendus à 48 bits en dupliquant certains bits (chaque groupe de 4 bits est entouré de ses bits voisins, ce qui donne 8 groupes de 6 bits).
2. **XOR avec la sous-clé :** les 48 bits obtenus sont combinés par XOR avec la sous-clé K_i de 48 bits.

3. **Substitution (boîtes S) :** les 48 bits sont découpés en 8 blocs de 6 bits, chacun passant dans une boîte S différente qui le transforme en 4 bits (tableau non linéaire de 4 lignes et 16 colonnes). On obtient 32 bits en sortie.
4. **Permutation (boîte P) :** les 32 bits issus des boîtes S sont permutés selon une table fixe.

Contribution à la confusion et à la diffusion :

— La **confusion** (rendre complexe la relation entre la clé, le texte clair et le texte chiffré) vient surtout des **boîtes S**, seule étape non linéaire, et du **XOR avec la sous-clé**, qui lie la sortie à la clé.

— La **diffusion** (répartir l'influence de chaque bit sur de nombreux bits de sortie) vient de la **boîte E**, qui fait influencer certains bits sur deux boîtes S à la fois, et de la **boîte P**, qui disperse les 4 bits de sortie de chaque boîte S vers des boîtes S différentes au tour suivant. L'échange des moitiés propage ensuite cet effet d'un tour à l'autre (effet d'avalanche).

3 Exercice 3 — Bits de parité

3.1 Question 1

Pourquoi dit-on que DES utilise une clé de 64 bits, alors que seuls 56 bits sont effectivement utilisés ?

La clé DES est fournie sous forme de 8 octets, soit **64 bits**. Cependant, dans chaque octet, seuls 7 bits sont de vrais bits de clé et le 8e bit est un bit de parité. La première opération de l'algorithme de génération des sous-clés (permutation PC-1) supprime ces 8 bits de parité et ne conserve que $8 \times 7 = 56$ bits, qui servent à produire les 16 sous-clés. On parle donc de clé de 64 bits dans sa représentation, mais de clé effective de 56 bits.

3.2 Question 2

Quel est le rôle des 8 bits restants ?

Les 8 bits restants (positions 8, 16, 24, ..., 64) sont des **bits de parité** à **parité impaire** : chaque octet de la clé doit contenir un nombre impair de bits à 1. Leur rôle est la **détection d'erreurs** lors du stockage ou de la transmission de la clé (une erreur sur un seul bit d'un octet rend la parité fausse). Ils n'apportent aucune sécurité supplémentaire et ne participent pas au chiffrement.

3.3 Question 3

Que peut-il se passer si un bit de parité est modifié pendant la transmission de la clé ?

Si un bit de parité est modifié, l'octet concerné n'a plus un nombre impair de bits à 1 : le récepteur qui vérifie la parité **détecte l'erreur** et peut rejeter la clé ou en demander la retransmission. En revanche, comme les bits de parité sont supprimés par PC-1, **les sous-clés ne changent pas** : si le récepteur ne vérifie pas la parité, le chiffrement et le déchiffrement fonctionnent exactement comme avant. À l'inverse, si c'est un bit de clé (et non un bit de parité) qui est altéré, toutes les sous-clés sont différentes et le déchiffrement échoue.

4 Exercice 4 — Exemple simplifié de Feistel

Considérons un chiffrement de Feistel dont les valeurs sont :

$$L_0 = 1010, \qquad R_0 = 1100, \qquad K_1 = 1111, \qquad K_2 = 1001$$

et dont la fonction de tour est $f(R, K) = R \oplus K$.

4.1 Question 1

Calculez L_1 *et* R_1 *(chiffrement), puis* L_2 *et* R_2.

Tour 1 (avec $K_1 = 1111$) :

$$\begin{aligned} L_1 &= R_0 = 1100 \\ f(R_0, K_1) &= 1100 \oplus 1111 = 0011 \\ R_1 &= L_0 \oplus f(R_0, K_1) = 1010 \oplus 0011 = 1001 \end{aligned}$$

Tour 2 (avec $K_2 = 1001$) :

$$\begin{aligned} L_2 &= R_1 = 1001 \\ f(R_1, K_2) &= 1001 \oplus 1001 = 0000 \\ R_2 &= L_1 \oplus f(R_1, K_2) = 1100 \oplus 0000 = 1100 \end{aligned}$$

Le texte chiffré est donc $(L_2, R_2) = (1001,\ 1100)$.

4.2 Question 2

Montrez comment retrouver L_0 *et* R_0 *à partir de* L_2 *et* R_2 *(déchiffrement).*

On utilise $R_{i-1} = L_i$ et $L_{i-1} = R_i \oplus f(L_i, K_i)$.
Annulation du tour 2 (avec K_2) :

$$\begin{aligned} R_1 &= L_2 = 1001 \\ f(L_2, K_2) &= 1001 \oplus 1001 = 0000 \\ L_1 &= R_2 \oplus f(L_2, K_2) = 1100 \oplus 0000 = 1100 \end{aligned}$$

Annulation du tour 1 (avec K_1) :

$$\begin{aligned} R_0 &= L_1 = 1100 \\ f(L_1, K_1) &= 1100 \oplus 1111 = 0011 \\ L_0 &= R_1 \oplus f(L_1, K_1) = 1001 \oplus 0011 = 1010 \end{aligned}$$

On retrouve bien $(L_0, R_0) = (1010,\ 1100)$, le message initial.

4.3 Question 3

Répétez le processus sans inverser l'ordre des clés et observez ce qui se passe. Qu'est-ce que cela montre ?

Annulation avec K_1 en premier :

$$\begin{aligned} R'_1 &= L_2 = 1001 \\ f(L_2, K_1) &= 1001 \oplus 1111 = 0110 \\ L'_1 &= R_2 \oplus f(L_2, K_1) = 1100 \oplus 0110 = 1010 \end{aligned}$$

Puis avec K_2 :

$$\begin{aligned} R'_0 &= L'_1 = 1010 \\ f(L'_1, K_2) &= 1010 \oplus 1001 = 0011 \\ L'_0 &= R'_1 \oplus f(L'_1, K_2) = 1001 \oplus 0011 = 1010 \end{aligned}$$

On obtient $(L'_0, R'_0) = (1010,\ 1010)$, alors que le message initial était $(1010,\ 1100)$. Le déchiffrement est faux.

Conclusion : l'ordre des sous-clés est essentiel. Chaque tour de déchiffrement doit utiliser la clé du tour qu'il annule, car il doit recalculer exactement la même valeur $f(\cdot, K_i)$ que celle XORée au chiffrement. Avec le mauvais ordre, les XOR ne s'annulent pas et le résultat est incorrect. Ici L'_0 coïncide avec L_0 par hasard (valeurs sur 4 bits seulement), mais $R'_0 \neq R_0$.

5 Travail personnel — Analyse conceptuelle

5.1 Question 1

Expliquez avec vos propres mots la différence entre confusion et diffusion.

La **confusion** consiste à rendre la relation entre la clé et le texte chiffré aussi compliquée que possible : en observant le chiffré, on ne doit pas pouvoir remonter à la clé. Elle s'obtient surtout par des *substitutions* (par exemple les boîtes S), qui remplacent des bits par d'autres de façon non linéaire.

La **diffusion** consiste à répartir l'influence de chaque bit du texte clair (ou de la clé) sur un grand nombre de bits du texte chiffré, afin de cacher les régularités statistiques du texte clair. Elle s'obtient surtout par des *permutations* et des mélanges répétés sur plusieurs tours. Un bon chiffrement a un effet d'avalanche : changer un seul bit d'entrée modifie environ la moitié des bits de sortie.

5.2 Question 2

Identifiez quelles parties de DES assurent principalement la confusion et lesquelles assurent la diffusion.

- **Confusion :** les **boîtes S** (seule partie non linéaire de DES) et, dans une moindre mesure, le **XOR avec la sous-clé**, qui rattache la sortie à la clé.
- **Diffusion :** la **boîte d'expansion E** (un bit d'entrée influence plusieurs boîtes S), la **boîte P** (disperse les sorties de chaque boîte S vers des boîtes différentes au tour suivant) et l'**échange des moitiés** du réseau de Feistel sur les 16 tours.

Les permutations initiale (IP) et finale (FP) n'apportent pas de sécurité cryptographique.

5.3 Question 3

Pourquoi ces deux propriétés sont-elles essentielles pour résister aux attaques statistiques ou différentielles ?

Les **attaques statistiques** exploitent les régularités du texte clair (fréquences des lettres, motifs répétés). La **diffusion** fait disparaître ces régularités en les répartissant sur tout le bloc chiffré, si bien que le chiffré ne laisse plus apparaître la structure du clair. La **confusion** empêche ensuite de déduire la clé à partir de relations simples entre clair et chiffré : sans elle, on pourrait résoudre le système d'équations reliant clé et chiffré.

Les **attaques différentielles** étudient comment une différence choisie entre deux textes clairs se propage vers une différence entre les chiffrés, pour en déduire des informations sur la clé. Des boîtes S fortement non linéaires (confusion) rendent ces propagations difficiles à prévoir, et la diffusion étale les différences sur tous les bits après quelques tours, ce qui empêche de suivre un chemin de différences exploitable. Sans l'une des deux propriétés, le chiffrement présenterait une faiblesse exploitable : sans confusion, il serait presque linéaire et donc résoluble ; sans diffusion, chaque bit de sortie ne dépendrait que de quelques bits d'entrée, ce qui permettrait de les attaquer séparément.